DIGITAL TWIN ASSISTED DYNAMIC NETWORK SLICING USING SSL, TGNN AND MARL FOR AI-NATIVE 6G NETWORKS

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

by

Keerthivasan S V (CB.SC.U4AIE23037)
Krish S (CB.SC.U4AIE23040)
Sriranjana C (CB.SC.U4AIE23066)
Thrishala S N (CB.SC.U4AIE23072)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Digital Twin Assisted Dynamic Network Slicing using SSL, TGNN and MARL for AI-Native 6G Networks**" submitted by **Keerthivasan S V (CB.SC.U4AIE23037)**, **Krish S (CB.SC.U4AIE23040)**, **Sriranjana C (CB.SC.U4AIE23066)**, and **Thrishala S N (CB.SC.U4AIE23072)** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision in the Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Sundaresan S

Thesis Advisor

Department of Artificial Intelligence,

Amrita Vishwa Vidyapeetham, Coimbatore Campus, India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



DECLARATION

We, **Keerthivasan S V., Roll. No: CB.SC.U4AIE23037**, **Krish S., Roll. No: CB.SC.U4AIE23040**, **Sriranjana C., Roll. No: CB.SC.U4AIE23066** and **Thrishala S N., Roll. No: CB.SC.U4AIE23072** hereby declare that this thesis entitled "**DIGITAL TWIN ASSISTED DYNAMIC NETWORK SLICING USING SSL, TGNN AND MARL FOR AI-NATIVE 6G NETWORKS**" is the record of the original work done by us under the guidance of **Dr. Sundaresan S**, Assistant Professor, Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: **Signature of the Students**

Date:

COUNTERSIGNED

Dr. K.P Soman

Professor and Dean,

Department of Artificial Intelligence

Amrita Vishwa Vidyapeetham,

Coimbatore Campus,

India.

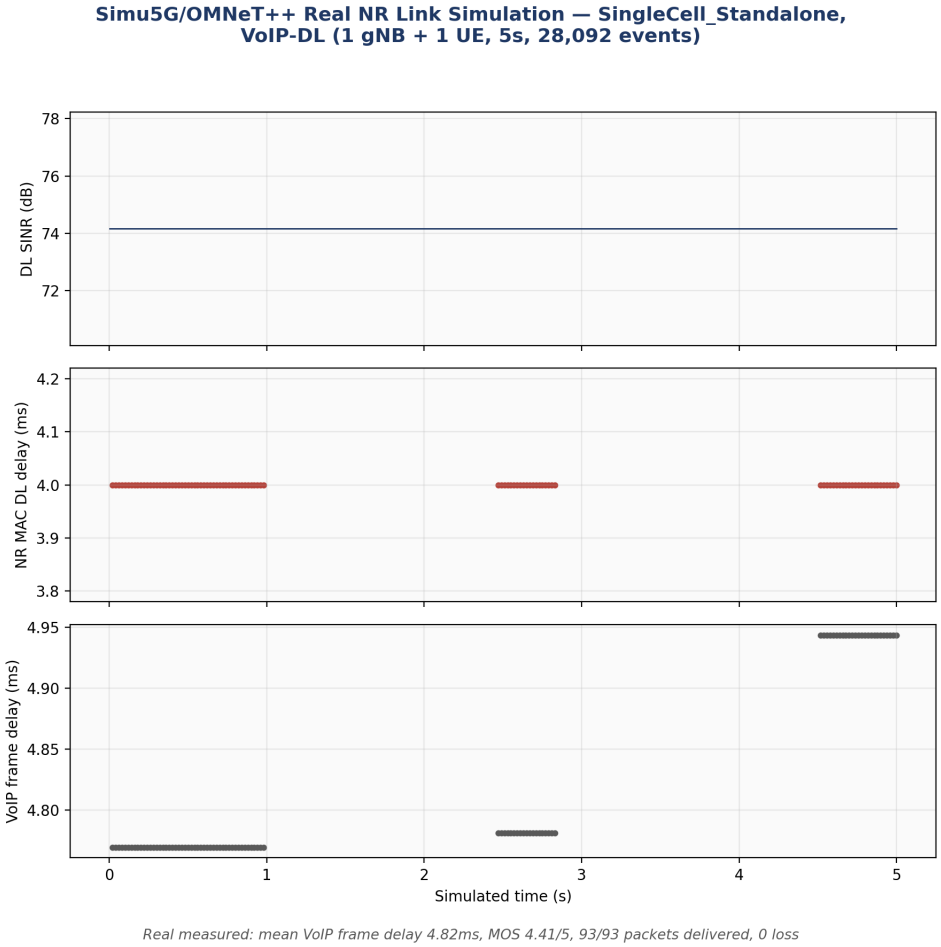


Figure 4.2: Reference Simu5G NR run (one gNodeB, one UE, 5 s, downlink VoIP)

4.8 Summary

The unified environment has real traffic and topology from Milan, slice demand, a latency and jitter model calibrated on 6G RAN telemetry, interference from neighbouring cells, and the reward of Equation (4.1). It allows forecasting and allocation to be tested together on the same traffic and compared with equal-split, proportional and expert policies, including tuned versions. Its main limit is the surrogate QoS model, which Chapter 8 compares with a packet-level simulator.

head; and logistic regression on the raw features together with the embedding. The metric is macro-averaged F1 over the nine classes.

Table 5.1: Macro-F1 on 81,731 unseen 5G-NIDD flows, by number of labelled flows (mean of five label draws up to 5,000 labels, two above)

Labels	LGBM raw	MLP raw	Logit raw	Masked emb.	Masked fine-tuned	SCARF emb.	SCARF fine-tuned	Raw + SCARF
50	0.643	0.694	0.704	0.627	0.684	0.594	0.688	0.690
200	0.852	0.858	0.853	0.763	0.834	0.713	0.850	0.848
1,000	0.905	0.895	0.874	0.859	0.881	0.842	0.894	0.888
5,000	0.919	0.911	0.909	0.901	0.898	0.880	0.901	0.915
20,000	0.932	0.924	0.923	0.913	0.923	0.907	0.925	0.924
all	0.936	0.925	0.926	0.912	0.927	0.910	0.929	0.926

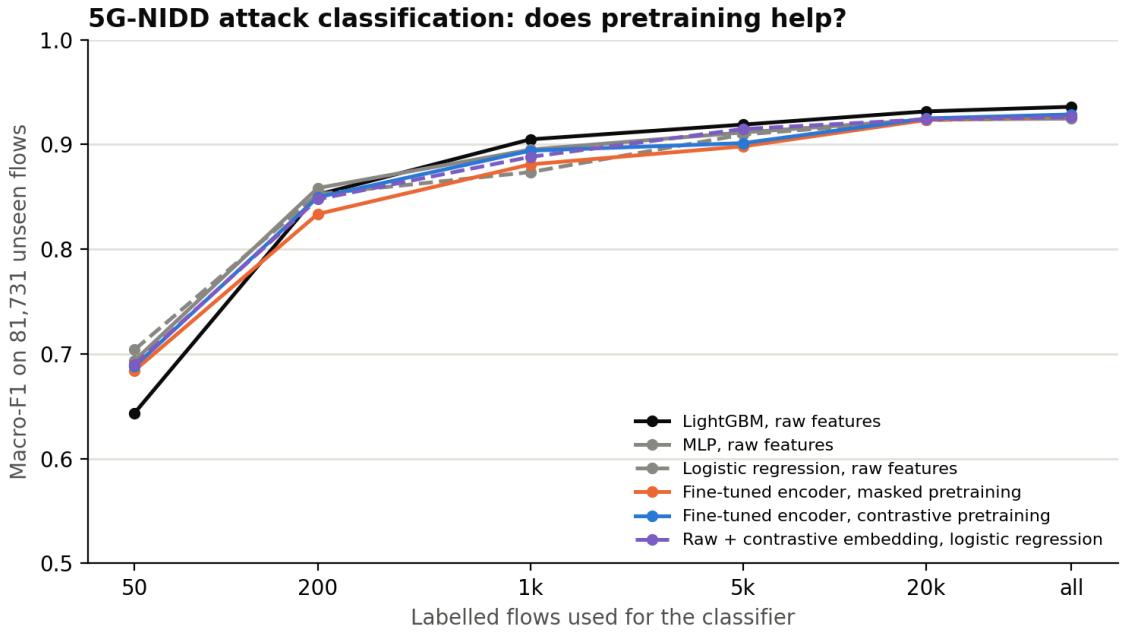


Figure 5.1: Macro-F1 against the number of labelled flows for raw features and pretrained encoders

Three observations follow from Table 5.1 and Figure 5.1.

1. The raw features are already strong. With all labels, logistic regression reaches 0.926 and LightGBM 0.936, compared with 0.708 in our first evaluation. Most of the earlier gap therefore came from duplicates and leaked identifiers, not from the classifier.

2. Pretraining does not beat the best raw-feature model at any budget. With 50 and 200 labels a raw-feature perceptron or logistic regression is ahead of the fine-tuned encoder (for example 0.694 and 0.704 against 0.688 at 50 labels), and a linear classifier on the frozen embedding alone is clearly worse. With all labels the fine-tuned contrastive encoder ties the same network trained from scratch (0.929 against 0.925) and stays below LightGBM.

3. Adding the embedding to the raw features helps a little, by at most 0.014 at 1,000 to 5,000 labels, and not at all with 20,000 or more.

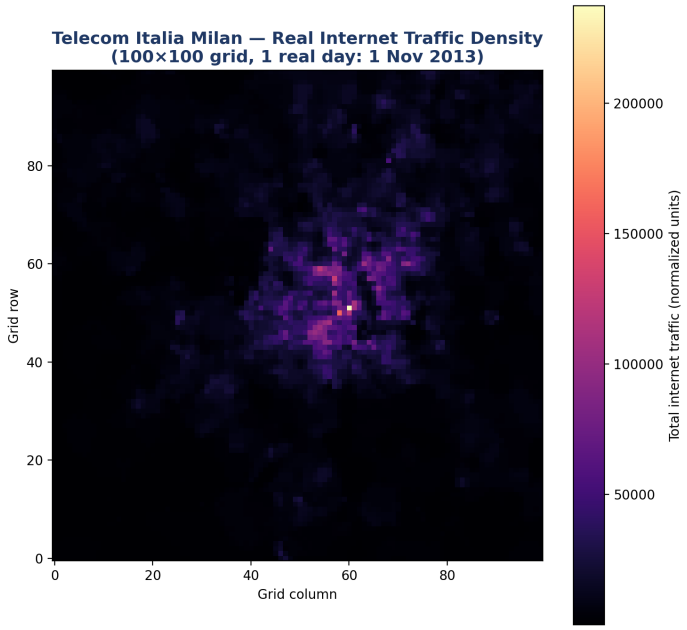


Figure 6.1: Real traffic on the Milan grid

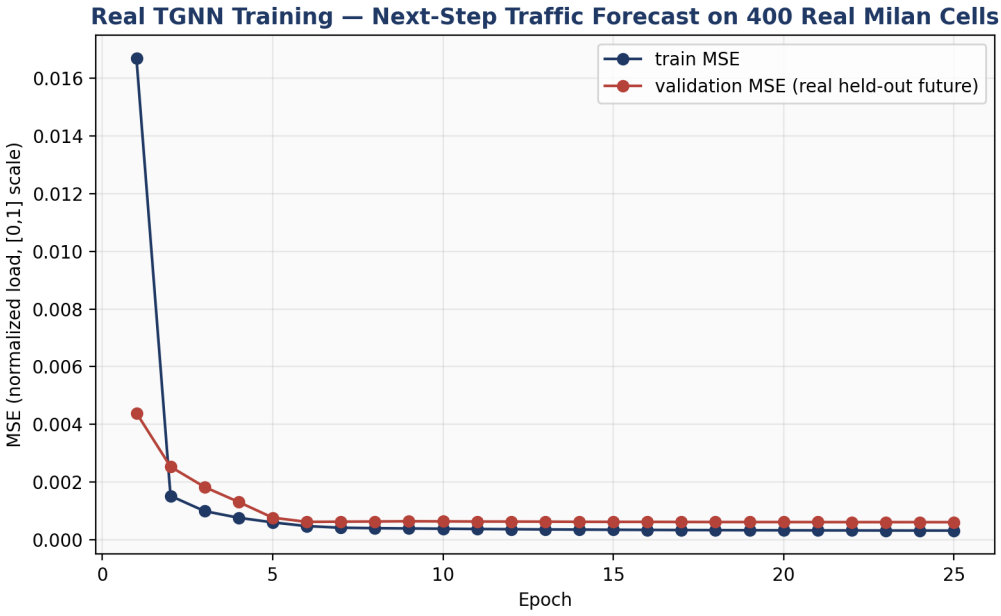


Figure 6.2: Training and validation error of the first Milan forecaster

Table 6.2: City-wide evaluation of the forecaster trained on the 100 central cells only (eMBB mean squared error, Mbps^2, two held-out days)

Region	**TGNN**	**Persistence**	**TGNN error relative to persistence**
100-cell block	54.00	61.23	0.9×
300-cell core ring	61.59	57.10	1.1×
9,600 outer cells	47.54	4.38	10.8×
All 10,000 cells	48.03	6.53	7.4×

1. **Per-cell scaling.** We divide the demand of each cell by its own typical level (the median over the training days of its daily 95th percentile, with a floor of 1% of the city-wide value). The model then learns the shape of the change and not the level.

2. **Spatial sampling.** Every training step uses a randomly placed 32×32 patch of the grid, so central, suburban and intermediate traffic are all seen.

3. **More data and better inputs.** The model trains on 39 real days. Days 5 and 6 are left out because we use them later for tests, and days 41 to 44 serve as validation for choosing the epoch. Each step also sees the demand of the cell at the same time on the previous day, which is known in advance, plus a time-of-day encoding and a weekend flag.

4. **No output ceiling.** The old output layer passed the mMTC prediction through a sigmoid, which capped it at the usual level of the cell. The new head predicts a correction to the last observed demand with no upper limit, so spikes can be forecast.

We test the model on a held-out week, 16 to 22 December 2013 (days 45 to 51), over every cell and against persistence. Table 6.3 gives the improvement in mean squared error.

Table 6.3: Improvement over persistence on the held-out week, 16–22 December 2013 (eMBB / URLLC / mMTC)

Region	Trained on 5 days	Trained on 39 days
100-cell block	-12% / +17% / +21%	+21% / +36% / +35%
300-cell core ring	-49% / +8% / +7%	+15% / +33% / +32%
9,600 outer cells	-34% / +9% / +5%	+18% / +34% / +31%
All 10,000 cells	-34% / +9% / +7%	+18% / +33% / +31%

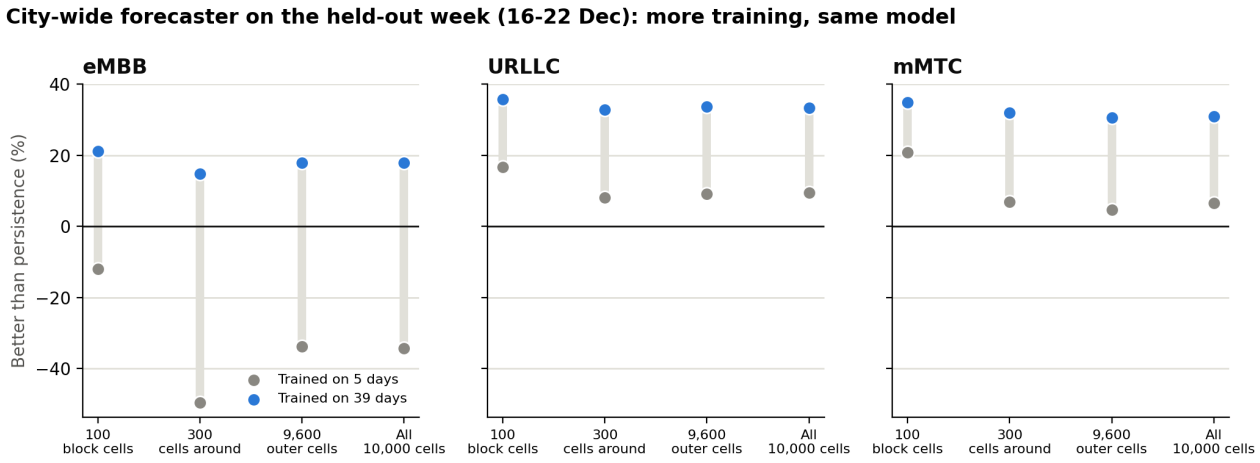


Figure 6.3: City-wide forecaster on the held-out week: five days of training against 39 days

The retrained forecaster beats persistence in every region and for every slice, including the 100 central cells where the earlier model was trained. The five-day version of the same model

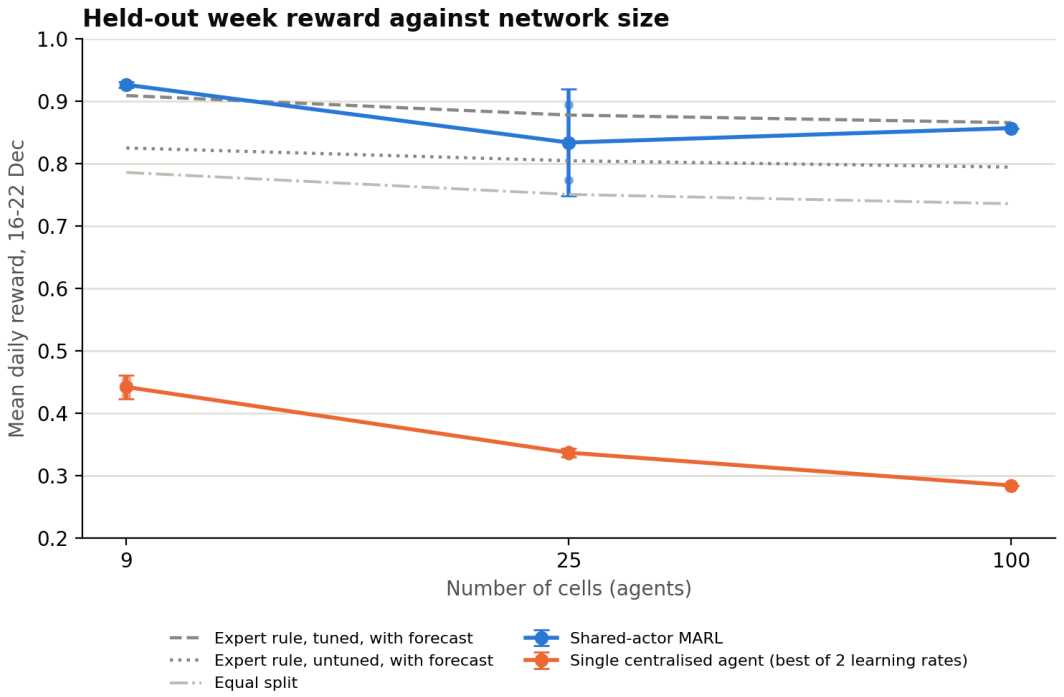


Figure 7.1: Reward against network size: shared MARL, the single agent and the reference rules

at a target, and the rest to eMBB. Its default targets were picked by hand, so we built two tuned versions that use the training days only.

- **Simple tuned rule.** Two utilisation targets, one for URLLC and one for mMTC, are chosen by grid search on the training days. At 25 cells the search picks 0.4 and 0.4, and the held-out reward of the forecast-fed rule rises from 0.804 to 0.878.

- **Rich tuned rule.** The targets are chosen separately for each four-hour block of the day and each of three cell classes (by traffic level), 18 pairs in total.

Table 7.1 shows that at 1,000 episodes the shared MARL controller is clearly better than the untuned rules, but at 25 and 100 cells it loses to the simple tuned rule. We therefore moved to a longer protocol: 4,000 episodes, the checkpoint with the best validation reward, and an average of the actors from several seeds in logit space. Table 7.3 gives the result.

Against the simple tuned rule, the learned controller is better on each of the seven test days at 9, 25 and 100 cells, by 0.023 to 0.024 in reward. Against the rich tuned rule the difference is about 0.005 or less, and at 25 and 100 cells the intervals include zero. So the off-policy controller matches a carefully tuned rule that knows the time of day and beats a simply tuned one, but it does not clearly beat the strongest hand-built rule. The on-policy learners of Section 7.7 do go beyond it. The 100-cell result rests on one seed, and its interval only describes the seven days.

Table 7.3: Long protocol (4,000 episodes, checkpoint chosen on validation days, actors averaged in logit space) against the tuned expert rules, held-out week (difference in reward, 95% CI, days better out of 7)

Cells	MARL (actors)	Simple tuned expert	MARL minus simple	Rich tuned expert	MARL minus rich	True-next oracle
9	0.933 (3)	0.909	+0.024 ± 0.004 (7/7)	0.928	+0.005 ± 0.004 (6/7)	0.945
25	0.904 (3)	0.877	+0.026 ± 0.005 (7/7)	0.899	+0.005 ± 0.003 (7/7)	0.920
100	0.888 (1)	0.865	+0.023 ± 0.006 (7/7)	0.887	+0.001 ± 0.003 (3/7)	0.908

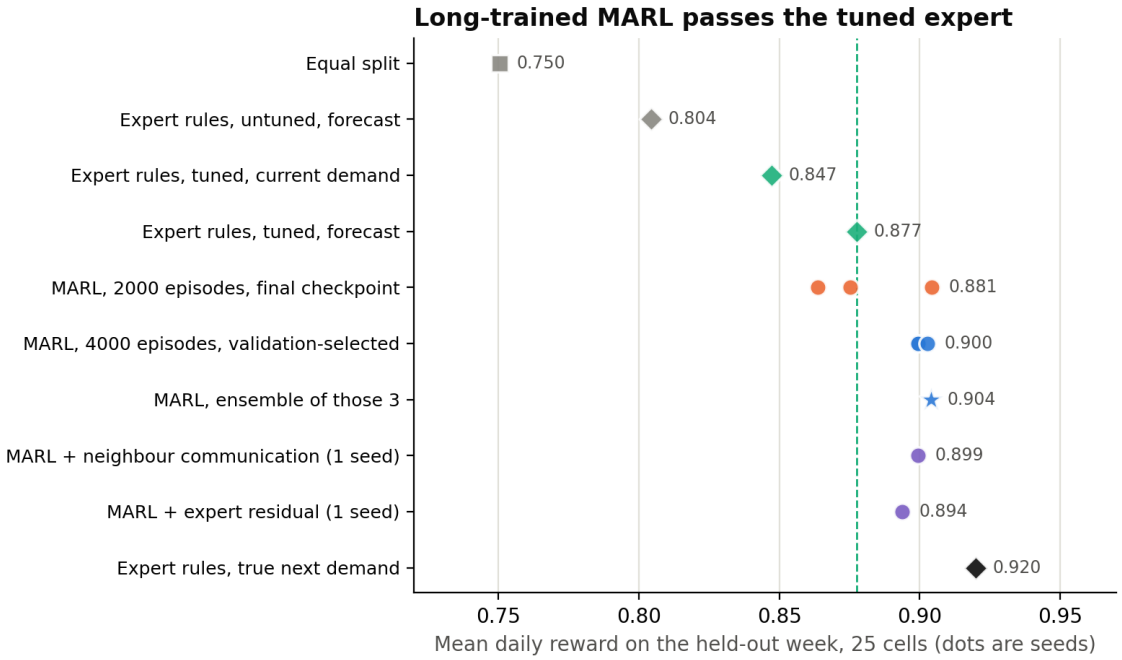


Figure 7.2: Long-protocol MARL against the tuned rules on the held-out week

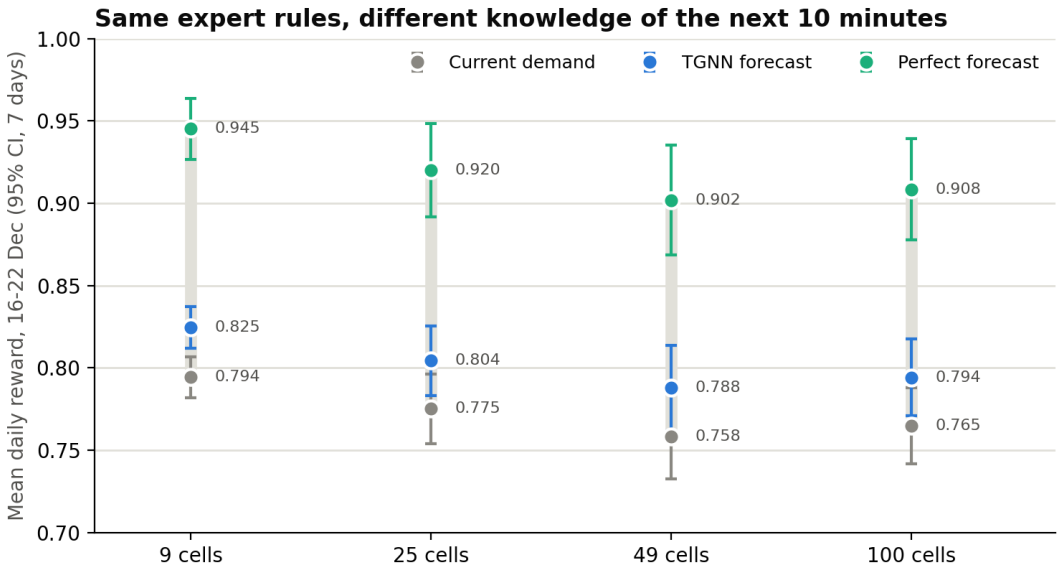


Figure 8.1: Value of prediction for the expert rule at four network sizes

The forecast improves the expert rule by about 0.03 in reward at every network size and on all seven test days, with intervals of about 0.004 to 0.008. It closes about one fifth of the gap to perfect knowledge. An earlier estimate of 0.05 came from two test days and a forecaster trained on five days. With 39 training days and a whole week of testing, the benefit is smaller but better established. Using the value of the previous day at the same time as a forecast does not work: its mean reward is 0.74 to 0.78 at every size, which is below that of current demand.

8.3 The Forecast and the Learned Controller

We also gave the forecast to the MARL controller as an extra input. Table 8.2 shows that at this training budget, with three seeds, it made no reliable difference. Giving the controller the true next-interval demand did not help either.

Table 8.2: MARL with and without forecast input (25 cells, three seeds, mean ± sample standard deviation)

Input to the MARL controller	**Held-out reward**
No forecast	0.778 ± 0.048
Forecast from the first (5-day) TGNN	0.765 ± 0.051
Forecast from the retrained TGNN	0.772 ± 0.073
True next-interval demand	0.762 ± 0.057

The three rewards differ by less than their sample standard deviations of 0.05 to 0.06, so the experiment shows neither a clear gain nor a clear loss. Under shifted traffic the effect on the learned policy even changes sign (Section 7.8). We therefore claim that forecasting helps the rule-based controller, and we have not shown that it helps the learned one. The forecast is

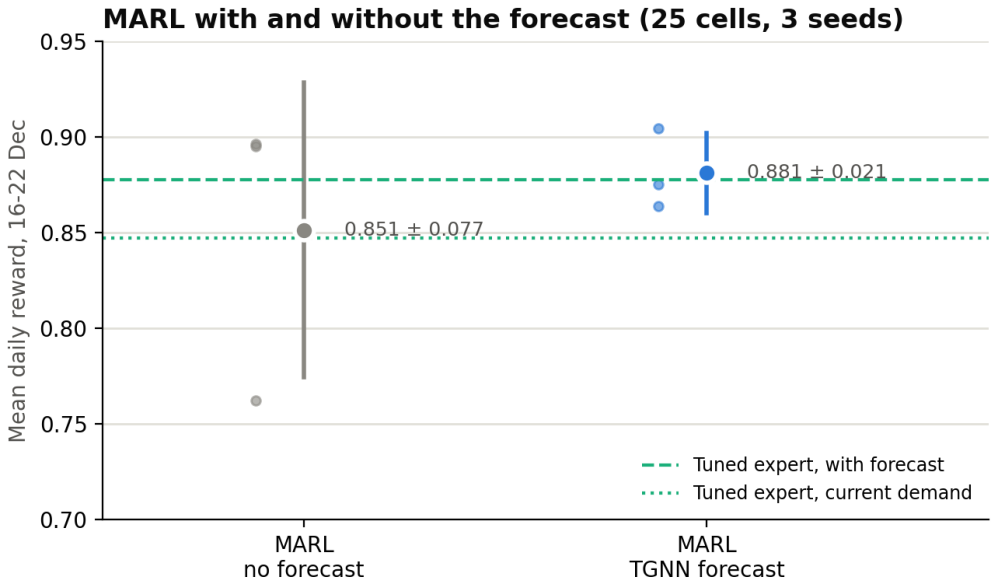


Figure 8.2: Learned controller with and without the forecast

still useful: the Digital Twin gate of Section 8.7 uses it to predict the outcome of a proposed allocation.

8.4 Controllers Across the Whole City

We ran the three controllers of the Slice Twin monitor (Chapter 9) on all 10,000 Milan cells for one day, Wednesday 6 November 2013, with scripted incidents. Static divides the capacity equally. Reactive applies the expert rule to the current demand. Predictive applies it to the city-wide forecast. The forecaster was not trained on this day.

Table 8.3: Controllers on all 10,000 cells, Wednesday 6 November 2013 (reward of the whole city, the dense block and the outer city)

Controller	**SLA met (eMBB / URLLC / mMTC)**	**Whole city**	**Dense block**	**Outer city**
Static	0.919 / 0.997 / 1.000	0.974	0.709	0.977
Reactive	0.975 / 0.711 / 0.568	0.839	0.735	0.840
Predictive	0.975 / 0.786 / 0.601	0.868	0.773	0.869

In the dense block the Predictive controller scores highest (0.773 against 0.735 and 0.709), as in Section 8.2. Over the whole city, however, Static scores highest (0.974 against 0.868 and 0.839). Most cells are lightly loaded, and when URLLC and mMTC traffic is close to zero the rule gives those slices very little capacity, so they miss their targets when traffic comes back. The outer city has most of the cells, and this outweighs the benefit of forecasting. An earlier run reported 0.935 for the Predictive controller over the city. That came from over-allocation

8.8 Checking the Surrogate Against a Packet-Level Simulator

The QoS model of our environment is a calibrated surrogate. To test it against a packet-level simulator we ran a load sweep in Simu5G 1.7.0 (OMNeT++ 6.4, INET 4.7), with one gNodeB and ten UEs receiving downlink UDP traffic in messages of 1000 bytes. The offered load went from 11% to 125% of the measured capacity of 56.1 Mbps, with three repetitions per point. Table 8.9 and Figure 8.3 show the delay.

Table 8.9: Simu5G load sweep (10 UEs, one gNodeB, downlink capacity 56.1 Mbps, 1000-byte messages, 6 s, mean of three repetitions)

Utilisation	Median delay (ms)	95th-percentile delay (ms)	Received (Mbps)
0.11	4.5	5.0	6.0
0.21	4.5	4.9	12.0
0.36	4.4	5.0	20.0
0.54	4.5	5.0	30.0
0.68	4.7	6.9	38.0
0.78	4.6	6.8	44.0
0.86	6.0	6.8	48.0
0.93	6.1	434.5	50.5
1.00	5.9	1376.2	50.1
1.07	5.8	780.8	50.7
1.25	11.1	211.9	55.6

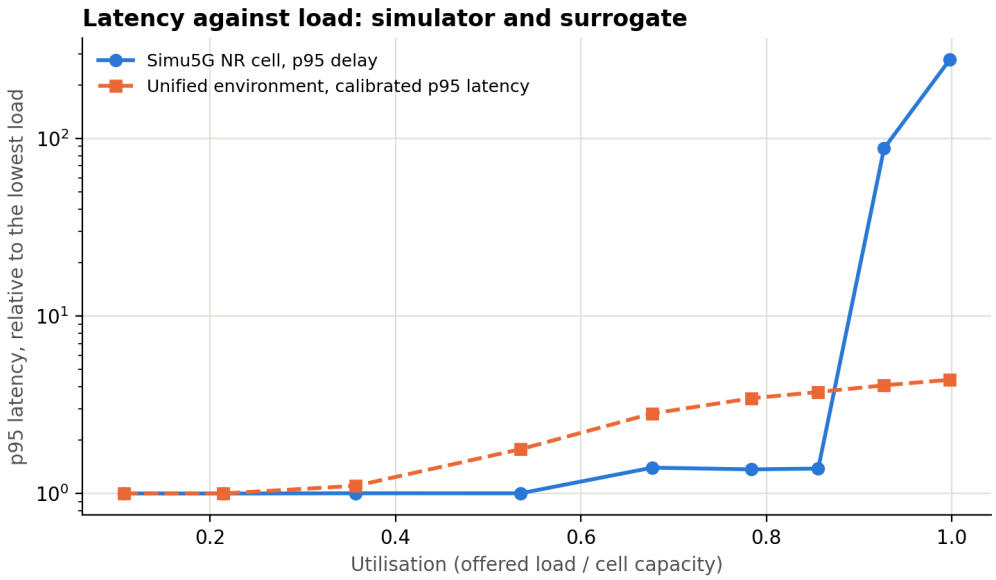


Figure 8.3: Delay against load: Simu5G and the surrogate of the environment (normalised)

The two models agree on the ordering: the rank correlation of the normalised 95th-percentile delay between Simu5G and our surrogate is 0.93. They do not agree on the shape. In Simu5G the delay stays flat at about 5 ms until the load reaches about 93% of capacity, and then it jumps, to 434 ms at 93% and above one second at full load. Our surrogate rises slowly from

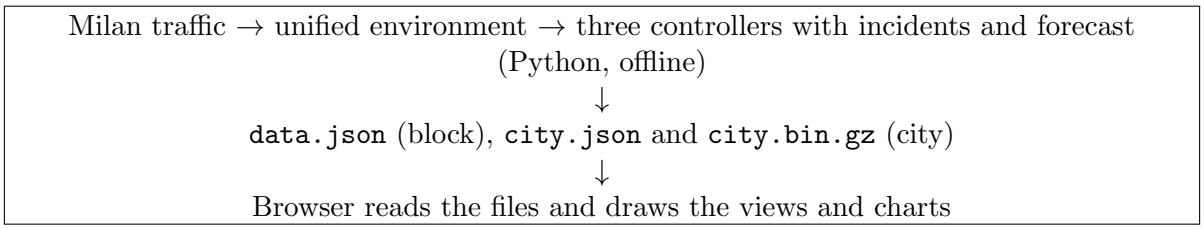


Figure 9.1: Data flow of the Slice Twin monitor

only reads the prepared files and draws the views (Figure 9.1). The files are produced by `showcase.py` for the block view and `showcase_city.py` for the city view, both in `src/unified`. The city data is stored compactly. For each interval and cell, one byte holds the log-scaled demand. For each controller, one byte holds the load (scaled by 250) and one byte holds flags: bits 0 to 2 say whether each slice meets its SLA, bit 3 marks an event and bit 4 an outage. The compressed binary file is a few megabytes, so the browser can handle 10,000 cells over 144 intervals and three controllers.

How to run it. From the root of the repository, `python -m http.server 8765 --directory app/slice_twin` starts the monitor at `http://localhost:8765`. To regenerate the data, run `showcase.py`, then `forecast_city_v2.py` once to train the city forecaster, and then `showcase_city.py`; the Milan files must be on the project drive. A GitHub Actions workflow publishes the monitor, the results dashboard and the project brief to GitHub Pages whenever the site or monitor folders change.

Demonstration. Figure 9.2 shows the city view at 17:30 on the demonstration day. On the left the cells are coloured by real traffic, with the storm-damage incident in the south marked, and on the right by SLA misses under the Reactive controller.

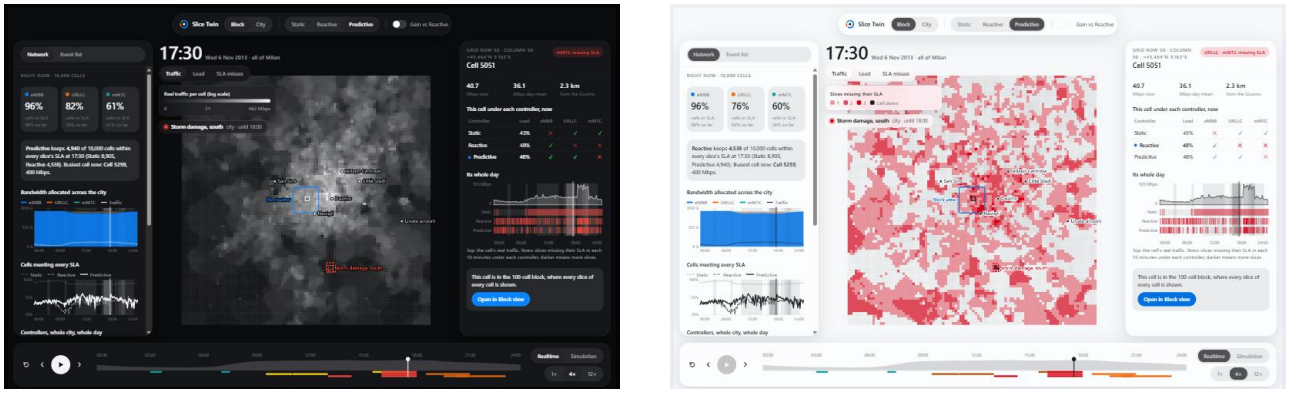


Figure 9.2: City view at 17:30: cells coloured by real traffic (left, dark theme) and by SLA misses under the Reactive controller (right, light theme).